

ML course season 2

Кластеризация. Часть II: DBSCAN и плотностной подход

4 мая 2026 г.

Содержание

1	Повторение: итоги первой части	1
2	Плотностная кластеризация: мотивация	1
2.1	Когда K-Means не справляется	1
2.2	Идея плотностного подхода	2
3	Алгоритм DBSCAN	2
3.1	Гиперпараметры и типы точек	2
3.2	Понятие плотной достижимости	2
3.3	Шаги алгоритма	3
4	Параметры eps и min_samples	3
4.1	Влияние eps	3
4.2	Влияние min_samples	3
4.3	k-Distance Plot	4
4.4	Чувствительность к масштабу	5
5	Работа с шумом	5
5.1	Что означает метка -1	5
5.2	Интерпретация в зависимости от задачи	5
5.3	Шум как диагностический сигнал	5
6	Ограничения DBSCAN и переход к HDBSCAN	6
6.1	Проблема кластеров с разной плотностью	6
6.2	HDBSCAN как решение	6
7	DBSCAN vs K-Means: сравнение	7
8	Практическое применение DBSCAN	8
8.1	Полный шаблон кода	8
8.2	Важные особенности реализации в sklearn	9
9	Итоги лекции	9

1. Повторение: итоги первой части

В первой части лекции по кластеризации мы разобрали задачу обучения без учителя и семейство алгоритмов K-Means. Напомним ключевые тезисы.

K-Means — итеративный алгоритм, чередующий два шага: назначение каждой точки ближайшему центроиду (Е-шаг) и обновление центроидов как среднего арифметического их кластеров (М-шаг). Алгоритм минимизирует внутрикластерное расстояние:

$$L(C, \mu) = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2 \rightarrow \min_{C, \mu}.$$

Мы также рассмотрели его варианты: **K-Means++** для умной инициализации, **Mini Batch K-Means** для ускорения на больших данных, **Bisect K-Means** для иерархического дробления.

Для оценки качества используются три метрики: внутрикластерное расстояние F_{in} (меньше — лучше), межкластерное расстояние F_{out} (больше — лучше) и коэффициент силуэта $S(x_i) \in [-1, +1]$ (больше — лучше).

Главное ограничение K-Means

Квадратичный штраф за расстояние до центра вынуждает алгоритм искать **выпуклые, шарообразные** кластеры примерно одинакового размера. Кластеры-«луны», кольца, спирали K-Means не может разделить принципиально.

2. Плотностная кластеризация: мотивация

2.1. Когда K-Means не справляется

Представим следующие сценарии:

- Данные о траекториях транспорта образуют изогнутые полосы, а не шары.
- В датасете транзакций есть небольшая группа аномально крупных операций (мошенничество).
- Мы хотим найти кластеры в 2D-облаке точек, не зная заранее их количество.

Во всех трёх случаях K-Means даёт неудовлетворительный результат. Нам нужен принципиально другой взгляд на понятие «кластер».

2.2. Идея плотностного подхода

Плотностное определение кластера

Кластер — это **максимальное множество точек, плотно связанных друг с другом**: любые две точки кластера соединены цепочкой, где каждое звено находится в достаточно плотной области пространства. Точки, не принадлежащие ни одной плотной области, считаются **шумом**.

Это определение не накладывает ограничений на форму кластера — оно позволяет находить произвольно вытянутые, изогнутые и вложенные структуры.

3. Алгоритм DBSCAN

3.1. Гиперпараметры и типы точек

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) задаётся двумя гиперпараметрами:

- ϵ (**eps**) — радиус окрестности: мы говорим, что точка q находится в ϵ -окрестности точки p , если $\rho(p, q) \leq \epsilon$.
- **min_samples** — минимальное число точек (включая саму точку), которые должны находиться в ϵ -окрестности, чтобы точка считалась «ядром».

На основе этих параметров каждая точка получает один из трёх статусов:

Типы точек в DBSCAN

- **Корневая точка** (core point): в её ϵ -окрестности не менее **min_samples** точек.
- **Граничная точка** (border point): в её ϵ -окрестности меньше **min_samples** точек, но сама она лежит в ϵ -окрестности какой-либо корневой точки.
- **Шумовая точка** (noise point): не является ни корневой, ни граничной. Получает метку -1 .

3.2. Понятие плотной достижимости

Плотная достижимость

Точка q непосредственно плотно достижима из корневой точки p , если q находится в ϵ -окрестности p .

Точка q плотно достижима из p , если существует цепочка точек $p = p_1, p_2, \dots, p_n = q$, где каждая следующая точка непосредственно плотно достижима из предыдущей (при условии, что все промежуточные точки — корневые).

Именно это понятие позволяет «тянуть» кластер вдоль произвольных форм: цепочка корневых точек может изгибаться как угодно.

3.3. Шаги алгоритма

1. Для каждой **непосещённой** точки найти её ϵ -окрестность.
2. Если число точек в окрестности $\geq \text{min_samples}$ — точка корневая, начинаем новый кластер.
3. **Расширяем** кластер: рекурсивно добавляем все плотно достижимые точки. Для каждой найденной корневой точки также проверяем её окрестность.

4. Граничные точки добавляются к кластеру, из корневой точки которого они достижимы.
5. Точки, не попавшие ни в один кластер — **шум** (метка -1).

Пример

Рассмотрим набор точек на плоскости. При $\text{eps}=0.5$, $\text{min_samples}=4$: точки в центре плотного скопления получают статус корневых; точки на границе скопления — граничных; несколько изолированных точек — шумовых. Алгоритм обойдёт все корневые точки и соберёт кластеры, не зная их количества заранее.

4. Параметры eps и min_samples

4.1. Влияние eps

Параметр ϵ определяет, что считать «соседями» точки. Его выбор критически важен:

- **eps слишком мало:** ϵ -окрестность большинства точек содержит мало соседей. Алгоритм объявляет большинство точек шумом и дробит настоящие кластеры на множество мелких.
- **eps слишком велико:** ϵ -окрестность охватывает почти все точки. Все кластеры сливаются в один, шумовые точки поглощаются.

4.2. Влияние min_samples

Параметр min_samples определяет, насколько плотной должна быть область, чтобы считаться «ядром» кластера.

- **Малые значения (2–3):** почти все точки становятся корневыми; шума очень мало, но и случайные группы из двух-трёх точек могут образовать «кластер».
- **Большие значения:** строже требования к плотности; больше шума, но кластеры соответствуют действительно плотным структурам.

Практическое правило: хорошая стартовая точка — $\text{min_samples} = 2 \times d$, где d — размерность данных. Для зашумлённых данных — увеличить. Значение меньше 3 обычно не имеет смысла.

4.3. k -Distance Plot

Наиболее распространённый способ выбора eps — *k-distance plot*:

1. Для каждой точки вычислить расстояние до её k -го ближайшего соседа (где $k = \text{min_samples}$).
2. Отсортировать эти расстояния по убыванию и построить график.
3. Найти «локоть» — точку резкого изгиба, где кривая переходит от крутого спуска к пологому.

4. Значение ϵ берётся приблизительно равным расстоянию в точке перегиба.

Интуиция: в плотном кластере расстояния до k -го соседа малы и примерно равны. Для шумовых точек эти расстояния значительно больше. «Локоть» отделяет одно от другого.

```
1  from sklearn.neighbors import NearestNeighbors
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  k = 5 # обычно равно min_samples
6  nbrs = NearestNeighbors(n_neighbors=k).fit(X)
7  distances, _ = nbrs.kneighbors(X)
8
9  k_distances = np.sort(distances[:, -1])[:, -1] # k-е расстояния
    ↪ по убыванию
10
11 plt.figure(figsize=(8, 4))
12 plt.plot(k_distances)
13 plt.xlabel("Точки (отсортированы)")
14 plt.ylabel(f"Расстояние до {k}-го соседа")
15 plt.title("k-Distance Plot – ищем «локоть»")
16 plt.grid(True)
17 plt.tight_layout()
18 plt.show()
```

4.4. Чувствительность к масштабу

Масштабирование обязательно

DBSCAN использует евклидово расстояние. Если признаки имеют разный масштаб — например, «возраст» (0–100) и «зарплата» (0–200 000) — расстояние будет определяться практически только одним признаком. Это делает параметр ϵ бессмысленным.

Решение: перед запуском DBSCAN всегда масштабировать данные, например StandardScaler.

```
1  from sklearn.preprocessing import StandardScaler
2  from sklearn.cluster import DBSCAN
3
4  scaler = StandardScaler()
5  X_scaled = scaler.fit_transform(X) # обязательный шаг
6
7  db = DBSCAN(eps=0.5, min_samples=5)
8  labels = db.fit_predict(X_scaled)
```

5. Работа с шумом

5.1. Что означает метка -1

В отличие от K-Means, DBSCAN явно выделяет точки, не принадлежащие ни одному кластеру. Они получают метку -1 . Это означает, что в ϵ -окрестности точки слишком мало соседей и она не достижима из плотных областей.

5.2. Интерпретация в зависимости от задачи

Важно понимать, что «правильная» интерпретация шума полностью зависит от контекста:

- **Очистка данных:** шумовые точки — ошибки измерения, артефакты, дубликаты. Их следует отфильтровать перед дальнейшим анализом.
- **Детектирование аномалий** (fraud detection, мониторинг оборудования): шумовые точки — это *самый ценный результат*. Именно они соответствуют аномальным событиям.
- **Сегментация:** если задача требует, чтобы каждая точка принадлежала кластеру, шумовые точки можно постфактум отнести к ближайшему кластеру.

5.3. Шум как диагностический сигнал

Доля шумовых точек — важный показатель правильности выбора гиперпараметров:

- **Шума слишком много** ($> 20\text{--}30\%$): `eps` слишком мало или `min_samples` слишком велико.
- **Шума нет вообще:** `eps` слишком велико или `min_samples` слишком мало.
- **Много шума несмотря на разумные параметры:** данные действительно содержат много аномалий — это важная находка.

```
1  # Анализ шумовых точек
2  n_noise = (labels == -1).sum()
3  noise_ratio = n_noise / len(labels)
4  print(f"Шумовых точек: {n_noise} ({noise_ratio:.1%} от
    ↳ выборки)")
5
6  # Вычленить координаты выбросов для анализа
7  X_noise = X[labels == -1]
8  print("Статистика выбросов по признакам:")
9  print(X_noise.describe())
```

6. Ограничения DBSCAN и переход к HDBSCAN

6.1. Проблема кластеров с разной плотностью

Ахиллесова пята DBSCAN — **единый глобальный порог плотности ε** . Рассмотрим набор данных с двумя кластерами: один очень плотный (точки расположены близко), другой разреженный (точки далеко друг от друга).

- При маленьком ε : алгоритм хорошо выделяет плотный кластер, но разреженный превращается в шум — точки не набирают достаточно соседей.
- При большом ε : разреженный кластер выделяется, но плотные кластеры поблизости сливаются.

Принципиальное ограничение

Для данных с разноплотными кластерами не существует значения ε , которое одновременно правильно выделяет все кластеры. Это не проблема настройки — это архитектурное ограничение алгоритма.

6.2. HDBSCAN как решение

HDBSCAN (Hierarchical DBSCAN) строит *иерархию кластеров*, рассматривая все возможные значения ε сразу. Затем выбираются наиболее «стабильные» кластеры — те, которые сохраняются в широком диапазоне ε .

Ключевые преимущества:

- Находит кластеры с разной плотностью в одном запуске
- Только один ключевой гиперпараметр — `min_cluster_size`
- Возвращает *вероятности* принадлежности к кластеру (мягкая кластеризация)
- Лучше работает в высокой размерности
- Поддерживает метод `predict` для новых точек (через приближение)

```
1  import hdbscan
2
3  clusterer = hdbscan.HDBSCAN(
4      min_cluster_size=10,      # минимальный размер кластера
5      min_samples=5           # влияет на «консервативность»
6      ↪ (опционально)
7  )
8  labels = clusterer.fit_predict(X_scaled)
9
10 # Дополнительная информация
11 probs = clusterer.probabilities_      # вероятности
    ↪ принадлежности [0, 1]
12 persist = clusterer.cluster_persistence_ # «стойкость»
    ↪ кластеров
```

Когда выбирать HDBSCAN вместо DBSCAN

Используйте HDBSCAN, когда: данные имеют разноплотные кластеры; размерность высокая; вам нужна мягкая принадлежность (вероятности); вы хотите избежать ручного подбора `eps`.

7. DBSCAN vs K-Means: сравнение

Критерий	K-Means	DBSCAN
Форма кластеров	только выпуклые	произвольная
Число кластеров	задаётся вручную (K)	определяется автоматически
Выбросы	нет обработки	явная метка <code>-1</code>
Вычислительная сложность	$O(N \cdot K \cdot T)$	$O(N \log N)$ с индексом
Разная плотность	плохо	плохо
Детерминизм	нет (зависит от <code>init</code>)	да
Масштабируемость	хорошо	умеренно
Нужно масштабирование	желательно	обязательно

Практическое правило выбора:

- Кластеры сферические, примерно одинакового размера, K известно — K-Means.
- Кластеры произвольной формы, есть выбросы, K неизвестно — DBSCAN.
- Кластеры разной плотности или высокая размерность — HDBSCAN.

8. Практическое применение DBSCAN

8.1. Полный шаблон кода

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from sklearn.cluster import DBSCAN
4  from sklearn.preprocessing import StandardScaler
5  from sklearn.neighbors import NearestNeighbors
6  from sklearn.metrics import silhouette_score
7
8  # 1. Масштабирование
9  scaler = StandardScaler()
10 X_scaled = scaler.fit_transform(X)
11
12 # 2. k-Distance Plot для выбора eps
13 k = 5
14 nbrs = NearestNeighbors(n_neighbors=k).fit(X_scaled)
15 distances, _ = nbrs.kneighbors(X_scaled)
16 k_dist = np.sort(distances[:, -1])[::-1]
```



```
17
18 plt.figure(figsize=(8, 3))
19 plt.plot(k_dist)
20 plt.title("k-Distance Plot")
21 plt.xlabel("Точки")
22 plt.ylabel("Расстояние до k-го соседа")
23 plt.grid(True)
24 plt.tight_layout()
25 plt.show() # Визуально определяем eps по «локтю»
26
27 # 3. Запуск DBSCAN
28 db = DBSCAN(eps=0.5, min_samples=5)
29 labels = db.fit_predict(X_scaled)
30
31 # 4. Анализ результатов
32 n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
33 n_noise = (labels == -1).sum()
34 noise_ratio = n_noise / len(labels)
35
36 print(f"Число кластеров: {n_clusters}")
37 print(f"Шумовых точек: {n_noise} ({noise_ratio:.1%})")
38
39 # Силуэт считается только по некластерным точкам
40 if n_clusters > 1:
41     mask = labels != -1
42     score = silhouette_score(X_scaled[mask], labels[mask])
43     print(f"Коэффициент силуэта: {score:.3f}")
44
45 # 5. Маски для визуализации
46 core_mask = np.zeros(len(labels), dtype=bool)
47 core_mask[db.core_sample_indices_] = True
48 border_mask = (~core_mask) & (labels != -1)
49 noise_mask = labels == -1
```

8.2. Важные особенности реализации в sklearn

- **Нет метода `predict`**: DBSCAN трансдуктивен — он работает только с теми данными, на которых обучался. Для предсказания на новых точках нужно запускать алгоритм заново или использовать HDBSCAN.
- **`db.core_sample_indices_`**: индексы корневых точек в обучающей выборке — полезно для визуализации.
- **`db.components_`**: координаты корневых точек (удобная альтернатива).
- **Расчёт силуэта с шумом**: при наличии шумовых точек (-1) их нужно исключать из расчёта силуэта, так как у них нет кластерной метки.

9. Итоги лекции

Ключевые тезисы

1. DBSCAN определяет кластеры как **плотно связанные области** и находит их произвольной формы.
2. Два гиперпараметра: ε (радиус окрестности) и `min_samples` (порог плотности).
3. Для выбора ε используется **k-distance plot**: ищем «локоть» кривой.
4. Масштабирование данных перед DBSCAN **обязательно**.
5. Шумовые точки (метка -1) — либо выбросы, либо ценный сигнал, зависит от задачи.
6. При разноплотных кластерах DBSCAN ломается — используйте **HDBSCAN**.